

中国科学院大学

《咨询游戏》调研与思考报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术

一、问题与初步分析

问题：

我们在 C 语言大作业写过五子棋 AI，底层核心通常是基于 Minimax 搜索树和 Alpha-Beta 剪枝。在开发过程中，我们都会遇到一个痛点：评估函数全靠人工总结规则，写一堆 if-else 来判断活三、冲四并赋予分值。为了提高棋力，我们不得不加深搜索层数，但这会导致计算量呈指数级爆炸。而传统的冯·诺依曼架构 CPU，只能串行地去遍历这些树节点。在这种传统架构下，如果我们想让五子棋 AI 变得更强大，可以通过什么方法进行优化；用我们常用的笔记本，在每步棋最多十秒思考时，传统方法的极限可以达到多少层深度？进一步地：现在的顶尖棋类 AI（如 AlphaGo）所采用的算法在逻辑和硬件实现上与我们写的有什么不同？能否给出一个标准衡量：如果同样在我们的笔记本上运行这种算法，他的棋力会是我们写的传统方法的多少倍？

初步分析：

我们的传统方法是：评估是按照棋型手写规则，搜索是 minimax 搜索树，剪枝是 Alpha-Beta 剪枝

当前的顶尖棋类：评估往往是神经网络等直接给出分数，搜索方法可以是相对常见的 MCTS(蒙特卡洛树)，同时也可以 minimax 搜索树，只不过评估方法变了而已。

我需要解决的问题：1. 如何解决由于搜索层数加深，导致的计算量变大的问题，在传统冯诺依曼架构下；

2. 传统方法的极限可以达到多少深度；

3. 总结当前的棋类 AI 的训练、推理方法，在硬件和逻辑上有什么不同；

4. 衡量棋力的方法、评测基准

二、经典文献/棋类工程项目调研结果

当前的文献对于问题 1 和问题 3 给出了很好的回答(不过后续我将继续尝试进行我的思考和补充)。

(一)问题一

首先是问题 1。目前的传统引擎的五子棋的 state-of-art 项目应该是奕心，但是并非开源；而目前开源的 Rapfi (dhbloo,2026) 也被称作是很好五子棋项目。同时我参考了一些同学的代码 (ie. Chen,2025/Jiang,2023)，更好地了解大家用的方式(甚至经过了期末比赛的严格的验证，可信度较高)。同时，一些经典论文也提出了一些很好的方法。经过对仓库、论文的阅读总结，得到以下的优化方式：

1. 剪枝方法/阈值在不同情况下的改变，有的因为可能最关键，所以给予奖励，搜索更多；有的剪的更厉害，因为很可能不重要，只是粗略地看一看：

(1) 顶层根节点：

a. 在迭代加深(Iterative Deepening)的每轮迭代中，不使用 $(-\infty, +\infty)$ 全窗口，而是基于上一轮搜索的得分 V ，开辟一个极窄的估计窗口 $[V - \Delta, V + \Delta]$ 。如果搜索不幸失败，再动态扩大边界。这是为了利用历史结果，最大化触发 Alpha 或 Beta 剪枝的概率。

b. 算法假设第一个走法(通常是置换表推荐的最优步)是最好的(PV 节点)。对其进行全窗口搜索后，对剩下的所有兄弟节点，全部使用零窗口 $(\alpha, \alpha + 1)$ 进行试探性快速搜索(侦察步)。只有当试探搜索返回的值大于 α 时，才会对该节点用全窗口进行复搜，这样可以将关键的算力给在最优步上。

(2) 中高层节点：

a. 激进剪枝——空步剪枝：在当前节点允许对手连续走两步。如果即使让我方停一手，搜索返回的分数仍然能够打破 Beta 边界(Fail-High)，说明我方优势巨大，当前整个大分支直接剪掉。

b. 唯一走法扩展:当置换表(TT)里的推荐走法比其他所有走法的最高得分明显高出一个巨大的 Margin 时,认为这是“唯一的正解”。引擎会给予该分支额外的深度奖励(Depth + 1)。

c.IIR / IID(内部迭代减深/内部迭代加深):因为 Alpha-Beta 剪枝的特点,结点的排序对剪枝效率有较大影响(Knuth&Moore,1975),所以如果在某个节点上置换表失效,引擎不会盲目去试,而是先以较低深度限制(比如当前深度减半)快速搜索一下,目的是“生猜”出一个最优走法用于排序,随后再进行真正的全深度搜索。

(3). 后期走法过滤:

(1) 后期走法减深:在经过 MovePicker 排序后,排在靠后的走法大概率是烂棋。引擎会对这些“后期走法”施加惩罚,直接降低它们的搜索深度。只有当减深搜索意外触发了 Fail-High 时,说明它确实可能是潜在的好棋才会将其恢复到正常深度进行复搜。

(2) 走法计数剪枝:当某个节点生成的走法太多,且已经试过了前 N 个高分走法都没能打破 Alpha 边界时,算法认为剩下的低分走法也不大可能赢,直接截断不再搜索。

(3) 基于策略/平庸步剪枝:如果某些格子既没有形成任何棋型(Trivial),在历史表或神经网络的 Policy 评分中又低得可怜,直接将其从走法列表里剔除。

2. 置换表记录一些已经算过的棋局。

将局面的哈希值(关键字)、搜索深度、搜索分数和当前玩家(人类或者 AI)记录在置换表中。下次在和当前局面是同一个局面,并且当时搜索得不比现在浅,那这个缓存结果就可以直接用,从而提升效率。

3. 优先进行防守杀棋:当看见对方有成四/双三等危险棋时,将防守点筛选出来,作为后续进行排序选择的候选列表,排除其他的棋子的选择。

4.threat-space search, 威胁空间搜索 (Allis et al., 1993)

即搜索那些让对手防守不住的连环进攻,最终的一步导向直接胜利;进攻的棋子往往只和当前棋盘上的棋子/进攻序列中的棋子有关。作者通过对威胁搜索树进行构建,可以高效率探测出绝大部分的威胁序列;如果没有威胁序列,用证明数进行进一步搜索即可。这样可以极好地进行进攻。

总结

这些方法从排序子节点、存储结果、加强或者削弱剪枝、以及思路从算分转向算杀,可以降低 branch factor、提升搜索效率。

(二)问题 3

接下来是问题 3:AI 的训练、推理方法。

AI 使用的搜索方法主要是 MCTS,同时也有使用 Minimax 作为搜索的,所以搜索方法上,主要是多了前者作为方法。MCTS 为什么不适合应用在一般的引擎里呢?因为 MCTS 所需要的是概率型的:当前的胜率和负率,这个用神经网络可以直接预测,然而由传统引擎是几乎写不出来的。

而评估方法是主要的差别:AI 往往直接输出当前的胜率 (-1 到 1 之间)、当前不同选择的概率分布,然后根据这个评估进行 MCTS 搜索。

Alpha-zero(Silver et al., 2017) 主要使用残差神经网络,将棋盘的状态 s 输入神经网络,得到 (p,v) ,其中 p 是选择各个选择走棋的概率分布, v 是当前的胜率。在训练的时候, p 和 v 要求分别接近于 π 和 z ,其中 π 是根据当前迭代的神经网络评估的值进行蒙特卡洛搜索之后得到的概率分布,相当于要求神经网络看一眼棋局和进行深思熟虑得到的结果相似; z 是下到终盘的胜负率 (-1,+1),要求当前看了一眼基本可以判断之后的胜负率了。具体的损失函数如下:

$$l = (z - v)^2 - \pi^T \log p + c \|\theta\|^2$$

(概率分布之间是交叉熵算是,值之间是 L_2 损失)

Muzero(Schrittwieser et al., 2020) 使用更少的残差神经网络,和 Alpha-zero 不同的是,它不仅维护了预测函数来预测价值和策略,还维护了表示器(将真实情况表示到一个隐空间中)和动力学函数(根据当前的状态和动作,推演出下一步的动作和及时的奖励)。因为在 Alpha-zero 中,表示是直接的 $15 \times 15 \times 2$ 的张量,而下了某个地方之后下一步的变化也是可以直接得出的,然而 Muzero 是不仅仅局限在五子棋中的,当前状态的表示、下一状态的变化并非如此清晰了然。

表示函数(根据过去 $1 \dots t$ 时间步的状态表示的)

$$s_0 = h_\theta(o_1, \dots, o_t)$$

预测函数(得到价值、策略)

$$p_k, v_k = f_\theta(s_k)$$

动力学函数(动作和当前状态, 得到下一状态和奖励预测)

$$r_k, s_k = g_\theta(s_{k-1}, a_k)$$

其中, o_i 是过去的外部输入的直接的序列, s_i 是内部隐状态, r_i 是直接的即时奖励训练的损失是: $l(\theta) = l(\pi, p) + l(z, v) + l(u, r) + c\theta$

前两项和 Alpha-zero 差不多, 后一项 $l(u, r)$ 是让我推演的内部的即时奖励 r 和外部的真实奖励 u 相近, 从而让我内部的状态表示 s_0 之间的转换能够和外部的奖励相匹配。

这激发了思考: 合适的表示能够解决很多问题。之前是直接地用张量表示棋盘, 现在先将棋盘转换到了隐藏层, 从而让隐藏层之间的转换和真实的奖励函数相匹配。当然, 也有用二维数组来表示棋盘的。

同时, 也有使用遗传算法来进行选择的 (Wang & Huang, 2014)。思考 7 步的序列是生物个体(包括 AI 和人类序列), 适应度函数使用的是传统的棋型作为分数, 我的分数 + 我防止人类下这里的分数。

每一次利用适应度函数进行适者生存, 然后将生存下来的个体进行序列交叉(模拟染色体交叉), 序列中一些地方进行突变(模拟基因突变), 最后生存下来的, 就是能在 7 步之内做到分数最大化的个体。这个不是传统的强化学习/深度学习, 在这里作为趣味介绍。

总结:

逻辑上, 棋类 AI 在评估上是通过输入棋盘情况, 直接得到选择的概率分布、当前的胜率预测, 这是在大量的数据(可能是人类对决结果, 也可能是自我博弈得到的)中建立的直觉, 具有难解释性; 人类 AI 在评估中往往直接根据棋型得到结果, 有大量人类先验的知识。

硬件上, 传统棋类 AI 往往直接使用 CPU 来进行计算, 而深度学习/强化学习中的 AI 往往使用 GPU 来在训练中进行反向传播、在推理中使用矩阵乘法等, 不过也会使用 CPU 来进行蒙特卡洛搜索、模拟得出下一步的状态(如果需要的话)、数据加载和预处理等。

因为 CPU 擅长进行分支预测, 在 if-else 的分支指令中比较高效; 同时 CPU 能很好地管理栈帧(因为栈空间比较大), 同时拥有高速缓存, 而入栈出栈的数据的局部性很强, 所以递归中的数据几乎都会“命中”高速缓存, 让读写变得很快。所以 CPU 做搜索、做基本条件判断是很适合的。

但是矩阵运算、反向传播、计算图的计算很需要 GPU, 因为这些往往会落脚到矩阵运算上, 而矩阵的运算大多有海量、重复、且彼此独立的计算任务, 而 GPU 有很多简单但是海量的核心, 可以高效地进行并行计算, 从而高效进行矩阵运算。

三、 我的思路和创新点

1. 表示棋盘

将棋盘表示得更加简洁, 便于计算/存储; 或者表示得更贴近游戏本身。

比如将棋盘进行位存储, 将 225 个 1/0 表示成 4 个 64 位的数字, 用与或非运算来实现对棋型的判断。比如

```
m = black & (black >> 1) & (black >> 2) & (black >> 3) & (black >> 4);
```

可以用来进行是否有黑色五连的判断。这样, 棋盘的表示的存储空间得以节省, 同时位运算往往比遍历数组更快, 所以速度从表示上也会提升。

而 Muzero(Schrittwieser et al., 2020) 的表示也耐人寻味, 它对棋盘的表示, 使得内在预测的即时奖励 r_k 和真实的奖励 u_k 很近, 这里的 u_k 往往在五子棋中是输/赢中的 -1 和 +1。

这一思想启发我将神经网络表示引入传统搜索框架中。具体而言,可以使用一个神经网络编码器将原始棋盘数组转换为高维张量表示,作为局面的隐式特征表达。随后,在 Minimax 或 Alpha-Beta 搜索过程中,不再完全依赖人工设计的评估函数,而是使用神经网络输出的价值作为叶子节点的局面评估值。

也就是说,可以将传统搜索中的评估函数

$$V(s) = f_{\text{handcrafted}}(s)$$

替换为神经网络评估函数

$$V(s) = f_{\theta}(s),$$

其中 s 表示当前棋盘状态, f_{θ} 表示经过训练的价值网络。该网络可以输出当前局面对当前执棋方的胜率或价值估计。这样, Alpha-Beta 搜索仍然保留其强大的局部搜索能力,而神经网络则负责提供更高层次、更接近游戏语义的局面判断。

这种方法相当于将传统博弈树搜索和深度学习评估函数结合起来。一方面,搜索可以弥补神经网络单步判断可能不准确的问题;另一方面,神经网络能够替代复杂的人工模型打分规则,使评估函数具有更强的泛化能力。也可以进一步利用这个方法,为候选落子进行排序,从而提高 Alpha-Beta 剪枝效率。

2. 搜索重点

五子棋是 Threat-sequence 很重要的一个游戏,如果找到了很好的对方难以防守/打断的威胁序列,我们就可以轻松胜利。但是过去的论文 (Allis et al., 1993) 对威胁树需要进行很长的搜索,如果搜索不成功还会进行更加复杂的证明数。而 AI 棋类往往会让神经网络的“一步直觉”逼近蒙特卡洛树的“多步深思”。那么我也可以通过搜索/已有数据等获得威胁序列树的数据样本,让神经网络输出当前拥有威胁序列的概率 v 和下一步应该走哪里,在训练的时候,让 v 和下一步走哪里贴近是否有树/如果有树下一步走哪里,这样可以快速得到是否有威胁树。

而威胁树的查找比起传统的评估概率而言,更加具有可解释性、并且引入了先验的知识,同时对深度没有限制(它的能力至少可以达到和我们的数据中所有的深度的思考),甚至可能超过这个深度。

而这样的威胁树可以用在 Alpha-Beta 的排序中的优先级很高的一步(v 大的排前面),也可以用于评估函数中的很重要的一项(结合模型等一起判断)

3. 如果不借助 AI

如果不想借助神经网络来帮忙表示/帮忙评估,需要把评估函数写的很好。有一种思考是,观察棋型分数很高的那些棋型,提炼出来,作为可查的哈希表/逻辑判断规则,从而在评估中快速且深度地实现了判断。或者直接逻辑思考一些相对复杂但是更深的逻辑。但这种方法的泛化能力和上限通常不如神经网络。神经网络可以自动学习复杂棋型和隐含关系,但可解释性较差。

四、传统方法的极限深度,在 10 秒以内

如果想做到深度很深的话,10 秒以内,需要速度尽量快。这需要剪枝尽量地高效(每一层的节点比较少)、评估尽量地快(评估函数简单),同时尽量不要折损搜索的质量。

方法可以用 state-of-art 的开源的模型加入层数打印代码直接进行实验测评(比如 Rapfi);也可以查找相关论文,换算相同的计算能力后,得出在“我们常用的笔记本”上计算的速度情况。

但是事实上后者是没有的。所以我将尝试前者。事实和想象的不同。在由于有的时候深度优先,短短几秒之内,层数就可以达到 30 多层;然后再递归下一种可能性,再从第 0 层开始。情况如图:

```
MESSAGE RECURSION_DEPTH 22
MESSAGE RECURSION_DEPTH 23
MESSAGE RECURSION_DEPTH 24
MESSAGE RECURSION_DEPTH 25
MESSAGE RECURSION_DEPTH 26
MESSAGE RECURSION_DEPTH 27
MESSAGE RECURSION_DEPTH 28
MESSAGE RECURSION_DEPTH 30
MESSAGE RECURSION_DEPTH 31
MESSAGE RECURSION_DEPTH 32
MESSAGE RECURSION_DEPTH 33
MESSAGE RECURSION_DEPTH 34
MESSAGE RECURSION_DEPTH 35
MESSAGE RECURSION_DEPTH 36
MESSAGE RECURSION_DEPTH 37
```

图 1: Rapfi 的层数结果打印(设备: Intel(R) Core(TM) i9-14900HX 处理器(拥有 24 个物理核心和 32 个逻辑线程,基准主频 2.20 GHz,配备 32.0 MB L2 缓存和 36.0 MB L3 缓存)

五、 棋力评测标准寻找

这个相对简单,对各大论文的 Result 部分进行总结基本可以得到。要求是能够在传统引擎和 AI 引擎中进行比较。

方法: 1.ELO 棋力对比 (Elo,1978)。

给所有欲对比的 AI 给一个相同的初始分 R_i ,而后两两对战,更新各自的分数。

一次两两对战中,先算出下一次胜负的概率期望:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

然后根据期望和彼此的比赛结果,对分数进行更新:

$$R'_A = R_A + K \times (S_A - E_A)$$

其中 K 是常数, S_A 是 1,0.5,-1,对应着赢、平、输,当期望和实际差别越大,分数变化也越大。

最终比较多局后,分数也就趋向于收敛,可以得到彼此之间的相对棋力对比。

但是这种只是测评了棋力,没有固定时间。可以看彼此完成比赛的时候的思考时间,进行两种变量的对比。如果一种方式和另一种方式的硬件设备相同,思考时间相近,那么棋力可以很好地进行对比。如果一种方式比另一方式快棋力还好,那么这种方式肯定强。

或者比赛中用相同设备,并且限制时间,时间到了,各大搜索树 (MCTS/Minimax) 必须停止,给出策略。这样的 ELO 也是变量几乎唯一的。

2. 棋类 AI 的速度和参数规模、网络大小有关。如果和某一种 AI 进行对比,可以选择该种网络下的某一规模的棋类 AI,使得和欲对比的 AI 的思考时间差别很小,然后再看胜率,这也是可行的。

3. 深度是不可行的。因为搜索方法不同,并且神经网络对于当前的评估已经融合了一些关于深度的直觉,所以单纯看递归深度恐有些偏颇。

4. 如果是专门对于威胁序列进行了优化,可以设计一些专门的威胁序列测试题,看看各大引擎是否能够发现威胁序列的防守点。威胁序列可以逐步加深/范围加广,从而可以看出各自对于威胁序列的探测能力、思考的深度/广度的能力。

参考文献和工程仓库

- [1] CHEN Q. gomoku: A personal Gomoku game and AI project[CP/OL]. (2025)[2026-06-02]. <https://github.com/chen-qian2230911429/gomoku>.
- [2] JIANG J Y. 2023-fall-c-Renju: Ucas-C-Gobang-(Renju)[CP/OL]. (2023)[2026-06-02]. <https://github.com/jiuhao47/2023-fall-c-Renju>.
- [3] DUAN H B, HANG Z Y, et al. Rapfi: One of the strongest Gomoku/Renju playing engine[CP/OL]. (2025)[2026-06-02]. <https://github.com/dhbloo/rapfi>.
- [4] Allis, L. V., Herik, H. J., & Huntjens, M. P. (1993). Go-moku and threat-space search. Maastricht, The Netherlands: University of Limburg, Department of Computer Science.
- [5] Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial intelligence*, 6(4), 293-326.
- [6] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ...& Hassabis, D. (2017). Mastering the game of go without human knowledge. *nature*, 550(7676), 354-359.
- [7] Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., ... & Silver, D. (2020). Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839), 604-609.
- [8] Wang, J., & Huang, L. (2014, September). Evolving Gomoku solver by genetic algorithm. In 2014 IEEE workshop on advanced research and technology in industry applications (WARTIA) (pp. 1064-1067). IEEE.
- [9] Elo, A. E. (1978). The rating of chessplayers, past and present. (No Title).